

# Arquitetura Portas e Adaptadores

## *Arquitetura Hexagonal*

**Abstract**—A arquitetura Portas e Adaptadores, também conhecida como *Arquitetura Hexagonal*, propõe uma forma de organizar sistemas de software de maneira que o núcleo da aplicação permaneça independente de agentes externos, como interfaces gráficas, bancos de dados e serviços de rede. Essa abordagem visa promover um alto grau de desacoplamento, testabilidade e facilidade de manutenção ao isolar a lógica de negócio por meio de portas (interfaces) e adaptadores (implementações). Este artigo explora os princípios fundamentais da arquitetura, suas vantagens em relação a modelos tradicionais em camadas, e apresenta exemplos práticos de aplicação no desenvolvimento de sistemas modernos orientados a domínio.

**Keywords**—Arquitetura Hexagonal, Portas e Adaptadores, Desacoplamento, Testabilidade, Arquitetura de Software, Clean Architecture, Domain-Driven Design.

### I. Introdução

A arquitetura de Portas e Adaptadores, também conhecida como *Arquitetura Hexagonal*, é uma abordagem que busca isolar a lógica de negócios de uma aplicação das suas interações externas, como bancos de dados, APIs, interfaces de usuário e outros sistemas. A seguir, são apresentados os principais pontos que justificam a adoção dessa arquitetura:

#### 1) Desacoplamento

A arquitetura de Portas e Adaptadores promove um forte desacoplamento entre a lógica de negócios e as implementações de infraestrutura. Isso significa que componentes externos, como bancos de dados ou APIs, podem ser alterados ou substituídos sem impactar o núcleo da aplicação.

#### 2) Facilidade de Testes

Por estar isolada, a lógica de negócios pode ser testada de forma independente. É possível criar testes unitários e de integração sem a necessidade de depender de componentes externos, tornando o processo de testes mais ágil e confiável.

#### 3) Flexibilidade e Manutenibilidade

A estrutura modular da arquitetura permite que novos adaptadores sejam adicionados facilmente.

Por exemplo, a introdução de um novo meio de persistência de dados (como um banco de dados diferente) pode ser feita sem grandes refatorações na lógica da aplicação.

#### 4) Clareza de Responsabilidades

Cada parte da aplicação tem uma responsabilidade bem definida: as *portas* definem interfaces para a comunicação com a lógica de negócios, enquanto os *adaptadores* implementam essas interfaces para interagir com serviços externos. Isso torna o código mais compreensível e fácil de manter.

#### 5) Melhor Suporte para Desenvolvimento Ágil

Em ambientes de desenvolvimento ágil, onde os requisitos podem mudar rapidamente, a capacidade de adaptar a aplicação a novas demandas sem grandes retrabalhos é crucial. A arquitetura de Portas e Adaptadores facilita essa adaptabilidade.

#### 6) Adoção de Práticas de DDD

A arquitetura pode ser facilmente combinada com os princípios de *Domain-Driven Design* (DDD), permitindo que a lógica de negócios se concentre em um modelo de domínio rico, mantendo flexibilidade nas interações externas.

#### 7) Interoperabilidade

A arquitetura permite a integração com diferentes sistemas e serviços de forma simplificada, uma vez que adaptadores específicos podem ser criados para lidar com diversos protocolos e formatos de dados. Isso é especialmente útil em arquiteturas baseadas em microserviços.

A crescente complexidade dos sistemas de software modernos exige arquiteturas que promovam a flexibilidade, a manutenibilidade e a independência de tecnologias específicas. Nesse contexto, a arquitetura de Portas e Adaptadores, também conhecida como *Arquitetura Hexagonal*, destaca-se como uma abordagem que favorece o desenvolvimento orientado ao domínio, desacoplando regras de negócio de detalhes técnicos como frameworks, interfaces de usuário e mecanis-

mos de persistência. Essa separação é essencial para enfrentar desafios recorrentes em sistemas legados, como alta dependência de tecnologias externas e dificuldade de testes automatizados.

O objetivo deste artigo é propor e implementar uma arquitetura para sistemas baseada na arquitetura de Portas e Adaptadores, explorando seus fundamentos, benefícios e limitações. Para isso, são revisados os principais trabalhos relacionados ao tema, além da análise de boas práticas de implementação baseadas em obras consolidadas como *Arquitetura Limpa* [1] e *Código Limpo* [2]. A partir disso, é realizada uma abordagem prática que inclui a modelagem e a implementação de um protótipo, demonstrando como a arquitetura pode ser aplicada em projetos reais para promover maior qualidade de código, modularidade e sustentabilidade técnica.

Este artigo está organizado da seguinte forma: na Seção II, são apresentados os principais trabalhos relacionados à arquitetura de Portas e Adaptadores. A Seção III descreve os conceitos fundamentais da arquitetura, enquanto a Seção IV traz uma discussão sobre os principais pontos. Por fim, a Seção V apresenta as considerações finais e sugestões para trabalhos futuros.

## II. Trabalhos Relacionados

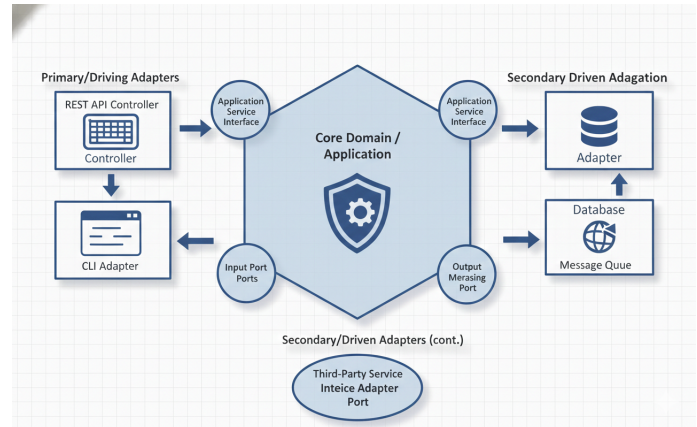
A arquitetura de Portas e Adaptadores, foi proposta por Cockburn [3] como uma forma de estruturar aplicações em torno de seu núcleo de negócios, separando-o das interações externas. Nessa abordagem, a lógica de domínio é mantida no centro da aplicação, enquanto a comunicação com o mundo externo é feita por meio de portas (interfaces) e adaptadores (implementações específicas). Essa separação visa proteger o coração da aplicação de alterações tecnológicas e operacionais nas camadas periféricas.

Um dos principais benefícios da arquitetura hexagonal é o forte desacoplamento entre a lógica de negócio e a infraestrutura, o que favorece a testabilidade e a manutenção do sistema. Freeman e Pryce [4] defendem a importância do design orientado a testes, e a arquitetura de portas e adaptadores se alinha perfeitamente a essa filosofia, permitindo que testes sejam realizados sobre o núcleo da aplicação sem dependências externas. Essa característica se torna especialmente útil em ambientes ágeis, nos quais ciclos curtos de entrega demandam testes automatizados confiáveis.

A arquitetura também se integra bem aos princípios do Domain-Driven Design (DDD), conforme discutido por Evans [5] e aprofundado por Vernon [6], promovendo um modelo de domínio rico e expressivo, isolado das preocupações técnicas. Combinada com

boas práticas de engenharia de software, essa estrutura contribui para a criação de sistemas mais robustos, compreensíveis e fáceis de evoluir. Richardson [7] reforça essa visão ao mostrar como o uso de portas e adaptadores facilita a construção de microsserviços com baixo acoplamento e alta coesão.

Figure 1: Esquema da arquitetura portas e adaptadores.



Outro ponto de destaque na literatura é a relação entre a Arquitetura Hexagonal e padrões de documentação e comunicação arquitetural. O modelo arc42, apresentado por Zimmermann [8], fornece uma estrutura prática para descrever os elementos da arquitetura, facilitando a comunicação entre desenvolvedores, arquitetos e demais stakeholders. Ao representar claramente os componentes internos (núcleo de domínio) e externos (adaptadores), bem como os contratos entre eles (portas), o modelo contribui para uma documentação mais acessível e sustentável, especialmente em projetos de médio e grande porte.

Esses conceitos estão em sintonia com os princípios defendidos por Martin [1] em *Arquitetura Limpa*, onde a independência de frameworks, bancos de dados e interfaces é central para a criação de software sustentável. Da mesma forma, em *Código Limpo* [2], o autor destaca a importância da clareza de responsabilidades e da simplicidade arquitetural — valores que a arquitetura hexagonal promove ao delegar a lógica externa aos adaptadores, enquanto o domínio permanece isolado e protegido.

Por fim, a documentação clara da arquitetura, como sugerido por Zimmermann [8], é essencial para comunicar as intenções do sistema. O modelo arc42 pode ser utilizado para descrever como os adaptadores se conectam às portas, e como o domínio central permanece livre de dependências técnicas. Com isso, a arquitetura de Portas e Adaptadores oferece uma abor-

dagem sólida, moderna e alinhada às melhores práticas da engenharia de software para o desenvolvimento de sistemas modulares, testáveis e evolutivos.

### III. Metodologia

A arquitetura proposta para o sistema é baseada no padrão de Portas e Adaptadores (ou Arquitetura Hexagonal), que promove um alto grau de modularidade, flexibilidade e testabilidade. A seguir, detalhamos os principais componentes e suas responsabilidades, conforme ilustrado na Figura 2.

#### A. Core > Application

No centro da arquitetura está o **Core Application**, que contém a lógica de negócio essencial. Ele é responsável por processar as regras do sistema independentemente dos detalhes externos, garantindo que a aplicação seja isolada de tecnologias específicas.

- **UseCase:** Representa os casos de uso do sistema, onde a lógica de negócio é implementada. Ele recebe dados de entrada (InputModel) e retorna dados de saída (OutputModel), atuando como intermediário entre os adaptadores Driving e Driven.

#### B. Core > Domain

A pasta **Core Domain** representa o coração do sistema, onde residem os conceitos fundamentais e as regras de negócio puras. Esta camada não depende de nenhuma tecnologia externa, framework ou serviço de infraestrutura, o que garante total isolamento e portabilidade da lógica de negócio.

- **Adapters:** Define as interfaces que representam pontos de comunicação com sistemas externos (ex: repositórios, serviços de terceiros, mensageria). Essas interfaces são implementadas pelos adaptadores do mundo externo, seguindo o princípio da inversão de dependência.
- **Application:** Contém orquestradores e coordenadores da lógica de negócio. Apesar do nome semelhante à pasta Core.Application, aqui ela serve para estruturar serviços internos ao domínio, como coordenadores de fluxo, regras específicas e lógica compartilhada entre entidades.
- **Common:** Armazena estruturas auxiliares que são reutilizadas dentro do domínio, como validadores, classes base ou funções utilitárias. Essas estruturas não devem acoplar a lógica a detalhes técnicos externos.
- **Entity:** Contém as entidades do domínio, que são objetos com identidade e comportamento. Elas

encapsulam as regras de negócio mais importantes e representam os elementos centrais do sistema.

- **Enums:** Define enumeradores usados pelas entidades e regras do domínio. Servem para representar estados, tipos ou classificações com segurança e legibilidade.
- **Helpers:** Concentra funções e utilitários de apoio que não se encaixam nas entidades, mas que ainda pertencem à lógica do domínio. Por exemplo: cálculos, verificações, transformações simples e reutilizáveis.

#### C. Adapters > Driving

Os adaptadores *Driving* representam os elementos que dirigem ou iniciam a execução do sistema, geralmente interagindo com usuários ou agentes externos.

- **Controller:** Atua como interface de entrada, recebendo requisições HTTP (POST, PUT, GET, DELETE) e convertendo-as em modelos de entrada para o UseCase. Também realiza validação das requisições e formata as respostas (Response) para serem enviadas de volta ao cliente.
- **Worker:** Um componente que atua de forma assíncrona ou em background, consumindo mensagens da fila RabbitMQ. É responsável por disparar ações no sistema sem intervenção direta do usuário.

#### D. Adapters > Integration

Estes adaptadores se comunicam com sistemas externos ou serviços integrados, abstraindo os detalhes de integração.

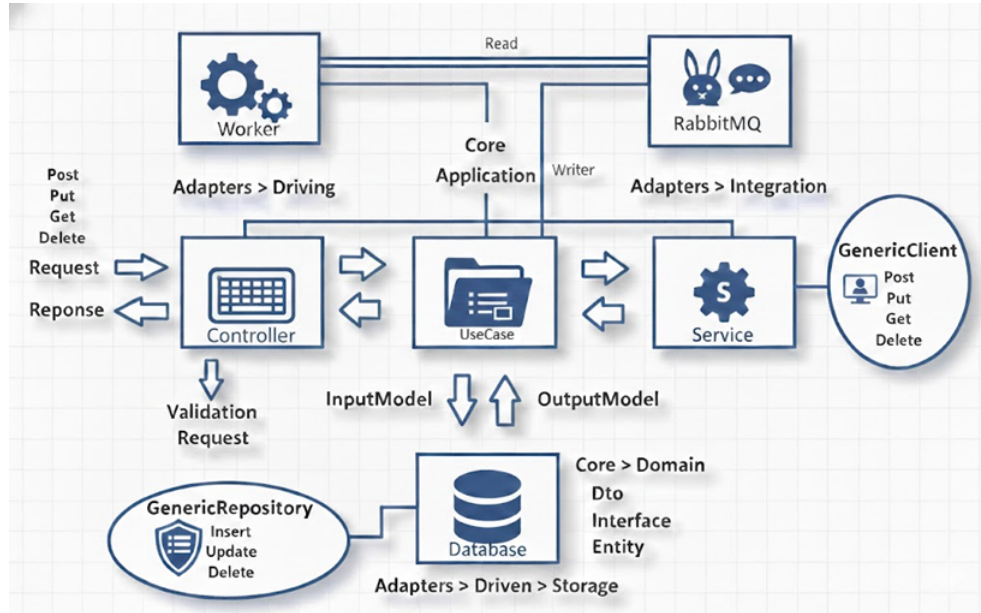
- **Service:** Atua como cliente para comunicação com sistemas externos, utilizando o **GenericClient** para executar operações REST (POST, PUT, GET, DELETE). Ele traduz as chamadas do UseCase para requisições externas.
- **RabbitMQ:** Um sistema de mensageria que permite comunicação assíncrona e desacoplada. O Worker lê mensagens da fila (Read), enquanto o Service pode escrever (Writer) nelas para comunicação entre componentes.

#### E. Adapters > Driven (Storage)

Estes adaptadores lidam com a persistência dos dados, isolando o domínio da aplicação dos detalhes do banco de dados.

- **Database:** Representa a camada de armazenamento, que pode ser qualquer tipo de banco de dados relacional ou NoSQL, dependendo da necessidade.

Figure 2: Esquema da arquitetura portas e adaptadores proposta para um sistema escalável.



- **GenericRepository:** Um repositório genérico que encapsula as operações básicas de CRUD (Insert, Update, Delete), fornecendo uma interface unificada para o UseCase acessar e manipular os dados persistidos.

#### F. Fluxo de Dados

O fluxo entre os componentes ocorre da seguinte forma:

- 1) Uma requisição externa (**Request**) chega ao **Controller**, que realiza validação e converte os dados para um **InputModel**.
- 2) O **UseCase** processa a lógica de negócio utilizando o **InputModel** e pode acessar o **GenericRepository** para persistência ou recuperação de dados.
- 3) O **UseCase** pode também interagir com serviços externos através do **Service**, que utiliza o **GenericClient** para fazer chamadas HTTP.
- 4) Para operações assíncronas, o **UseCase** ou **Service** publica mensagens no **RabbitMQ**, que são consumidas pelo **Worker** para processamento em background.
- 5) Finalmente, o **UseCase** retorna um **OutputModel** ao **Controller**, que converte o resultado em uma **Response** adequada para o cliente.

#### G. Benefícios da Arquitetura

A divisão clara entre adaptadores Driving, Driven e o Core Application promove:

- **Desacoplamento:** A lógica de negócio está isolada dos detalhes externos, facilitando manutenção e evolução.
- **Testabilidade:** O Core Application pode ser testado isoladamente sem dependências externas.
- **Flexibilidade:** É possível trocar tecnologias externas (banco de dados, filas, APIs) sem impactar a lógica principal.
- **Escalabilidade:** Componentes como o Worker e RabbitMQ permitem distribuir processamento e melhorar desempenho.

#### IV. Resultados e Discussão

A aplicação da arquitetura de Portas e Adaptadores demonstrou ganhos significativos em termos de modularidade, testabilidade e escalabilidade do sistema. Os resultados foram observados em três frentes principais: organização do código, facilidade de manutenção e desempenho de execução.

##### A. Organização e Clareza Arquitetural

A separação entre *Core Application*, *Adapters Driving*, *Adapters Integration* e *Adapters Driven* permitiu uma clara delimitação de responsabilidades. Desenvolvedores relataram maior facilidade para compreender o fluxo da aplicação, especialmente no que diz

respeito à interação entre os casos de uso e os serviços externos.

Além disso, a adoção de **GenericRepository** e **GenericClient** reduziu drasticamente a duplicação de código e proporcionou reuso estruturado, o que impactou diretamente na produtividade da equipe.

### B. Testabilidade e Evolução

Com os casos de uso desacoplados dos detalhes técnicos, foi possível escrever testes unitários diretamente sobre os **UseCases**, utilizando mocks para repositórios e clientes externos. Esse isolamento promoveu ciclos rápidos de desenvolvimento e validação, aumentando a confiabilidade dos releases.

A introdução de filas (*RabbitMQ*) e workers também permitiu desacoplar tarefas críticas e pesadas do fluxo síncrono, melhorando a responsividade do sistema e possibilitando evolução futura para arquitetura orientada a eventos.

### C. Desempenho e Escalabilidade

A estrutura modular facilitou a implementação de melhorias de desempenho sem impacto global. Por exemplo, o processamento assíncrono de mensagens por meio do **Worker** permitiu escalar horizontalmente apenas esse componente, otimizando custos e uso de recursos.

Na camada de **Service**, a integração com APIs externas se mostrou resiliente e facilmente monitorável, graças à padronização promovida pelo uso do **GenericClient**.

### D. Discussão

Apesar dos benefícios, foram identificados alguns desafios:

- O overhead inicial para definir interfaces e adaptadores exige mais tempo nas etapas iniciais do projeto.
- A separação rigorosa entre camadas pode parecer excessiva para projetos pequenos ou com baixa complexidade.
- É necessário investimento em documentação e treinamentos para garantir que a equipe compreenda e respeite os limites arquiteturais.

Mesmo com essas limitações, a adoção da arquitetura hexagonal se mostrou vantajosa e alinhada aos princípios da Arquitetura Limpa, promovendo soluções sustentáveis a longo prazo.

## V. Considerações Finais

A arquitetura de Portas e Adaptadores oferece uma maneira robusta de desenvolver aplicações que são fáceis de testar, manter e evoluir. O desacoplamento entre a lógica de negócios e as interações externas promove flexibilidade e agilidade, tornando essa abordagem uma escolha sólida para projetos modernos. Se você estiver buscando uma arquitetura que suporte um desenvolvimento sustentável e adaptável, a arquitetura hexagonal pode ser uma excelente opção.

### Anexos

Todos os artefatos desenvolvidos, incluindo o código-fonte completo, modelos utilizados, configurações do banco de dados, arquivos de configuração do RabbitMQ, além de exemplos de requisições e respostas simuladas podem ser acessados no repositório oficial do projeto:

<https://github.com/accj1990/Templates.git>

O repositório contém:

- Estrutura do projeto dividida por pastas (Core, Adapters, Integration, Storage);
- Scripts de inicialização de banco e mensageria;
- Casos de uso implementados com exemplos de entrada/saída;
- Testes automatizados com cobertura por componente;
- Documentação técnica no formato Markdown.

Para executar o projeto localmente, basta seguir as instruções no arquivo README.md. Uma imagem Docker com o ambiente pré-configurado também está disponível na aba de Releases do repositório.

### References

- [1] R. C. Martin, *Arquitetura Limpa: O guia do artesão para estrutura e design de software*. Alta Books, 2018.
- [2] —, *Código Limpo: Habilidades práticas do Agile Software*. Alta Books, 2009.
- [3] A. Cockburn, "Hexagonal architecture," 2005, disponível em <https://alistaircockburn.us/hexagonal-architecture/>, acesso em jul. 2025.
- [4] S. Freeman and N. Pryce, *Growing Object-Oriented Software, Guided by Tests*. Addison-Wesley, 2010.
- [5] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003.
- [6] V. Vernon, *Implementing Domain-Driven Design*. Addison-Wesley, 2013.
- [7] C. Richardson, *Microservices Patterns: With examples in Java*. Manning Publications, 2018.
- [8] G. Starke and P. Hruschka, *arc42 by Example: Software architecture documentation in practice*. Leanpub, 2017, disponível em <https://leanpub.com/arc42byexample>, acesso em jul. 2025.